```
my_computer = Computer.new(42, DS.new)
my_computer.cpu   # => * Cpu: 2.9 Ghz quad-core ($120)
```

It worked. Bill, however, is concerned about one last detail.

respond_to_missing?

If you specifically ask a Computer whether it responds to a Ghost Method, it will flat-out lie:

```
cmp = Computer.new(0, DS.new)
cmp.respond_to?(:mouse)       # => false
```

This behavior can be problematic, because respond_to? is a commonly used method. (If you need convincing, just note that the Computer itself is calling respond_to? on the data source.) Fortunately, Ruby provides a clean mechanism to make respond_to? aware of Ghost Methods.

respond_to? calls a method named respond_to_missing? that is supposed to return true if a method is a Ghost Method. (In your mind, you could rename respond_to_missing? to something like ghost_method?.) To prevent respond_to? from lying, override respond_to_missing? every time you override method_missing:

```
class Computer
  # ...

➤  def respond_to_missing?(method, include_private = false)
➤    @data_source.respond_to?("get_#{method}_info") || super
➤  end
end
```

The code in this respond_to_missing? is similar to the first line of method_missing: it finds out whether a method is a Ghost Method. If it is, it returns true. If it isn't, it calls super. In this case, super is the default Object#respond_to_missing?, which always returns false.

Now respond_to? will learn about your Ghost Methods from respond_to_missing? and return the right result:

```
cmp.respond_to?(:mouse)   # => true
```

Back in the day, Ruby coders used to override respond_to? directly. Now that respond_to_missing? is available, overriding respond_to? is considered somewhat dirty. Instead, the rule is now this: remember to override respond_to_missing? every time you override method_missing.

If you like BasicObject#method_missing, you should also take a look at Module#const_missing. Let's check it out.